PampelMuse

Sebastian Kamann (skamann@astro.physik.uni-goettingen.de)

October 24, 2014

Abstract

PampelMuse is a software package with the purpose of facilitating the analysis of integral-field spectroscopic observations of crowded stellar fields. It provides several subroutines to perform the individual steps of the data analysis. The theoretical work that went into the development of this code is described in detail in Kamann et al. (2013) and not repeated here. Instead, the purpose of this document is to provide a practical guide on the analysis of IFS data with PampelMuse .

1 Getting started

1.1 Requirements

PampelMuse is written in *python*. Currently version 2.7 of *python* must be available on the system (because earlier versions lack the `argparse` module that is required). Support for *python 3* is planned for future revisions. In addition, several *python* packages are required to run PampelMuse :

- *numpy*[1], tested for version 1.6.1
- *scipy*[2], tested for versions ≥0.9, a version ≥0.10 is recommended because a faster algorithm for sparse matrix inversion was introduced in v0.10
- *astropy*[3], tested for version 0.2.5

Note that the versions of the packages listed above are just a recommendation. The code may also run with earlier versions. In addition to these packages, it is highly recommended that the following packages are available on the system:

- *matplotlib*[4], tested for version 1.1.1
- *PyQt4*[5]

These packages are required in order to work with the graphical user interface (GUI) that is provided with PampelMuse . When starting PampelMuse the availability of *matplotlib* and *PyQt4* is checked. If the packages are not available, the interactive mode of will be disabled. Finally, the packages listed below are not required to run the core packages of PampelMuse , but some auxiliary routines depend on them:

- *astLib*[6], tested for version 0.7.0
- *pyraf*[7]

1.2 Download and Installation

There are currently two possibilities to obtain PampelMuse that will be described in the following.

[1] `http://www.numpy.org/`
[2] `http://scipy.org/`
[3] `http://www.astropy.org/`
[4] `http://matplotlib.org/`
[5] `http://www.riverbankcomputing.co.uk/software/pyqt/intro`
[6] `http://astlib.sourceforge.net/`
[7] `http://www.stsci.edu/institute/software_hardware/pyraf`

1.2.1 Using the tarball – TBD!

A zipped version of the code is available at `http://path.to/pampelmuse.tar.gz`. After the download is completed, change to the folder containing the tarball and execute the following commands

```
# tar -xvzf PampelMuse-0.17.1.tar.gz
# cd PampelMuse-0.17.1/
# python setup.py install
```

This will install PampelMuse to your default *python* path. Note that the last step usually requires root privileges. If you do not have them, you can define a custom path using the `--prefix` or `--home` arguments. For example, the following command will install PampelMuse to folders `$HOME/lib/python` and `$HOME/bin`:

```
# python setup.py install --home=~
```

Afterwards you have to make sure that the system finds the newly installed files by adding `$HOME/bin` to your `PATH` and `$HOME/lib/python` to your `PYTHONPATH`. For a bash shell, this is done by adding

```
# export PYTHONPATH=$PYTHONPATH:$HOME/lib/python
# export PATH=$PATH:$HOME/bin
```

to your shell configuration file (`$HOME/.bashrc`). For a c-shell, the following commands lines should be added to your configuration file (`$HOME/.cshrc`):

```
# setenv PYTHONPATH $PYTHONPATH:$HOME/lib/python
# setenv PATH $PATH:$HOME/bin
```

For further informations on custom installation paths, check the *python* documentation.[8]

Afterwards, you should be able to execute

```
# PampelMuse --help
```

If no error message is returned, you have installed PampelMuse successfully!

1.2.2 Using the SVN repository

The maintenance of PampelMuse is handled via an SVN repository located at `http://svn.astro.physik.uni-goettingen.de/pampelmuse`. The advantage of using the repository is that it allows you to always use the most recent version of the code. However, the disadvantage is that when updating the code I sometimes commit revisions that are not fully operational or have freshly introduced bugs. So when updating to a new revision, be sure to check the most recent log entries.

To install from the SVN repository, first cd into the folder were you want to install the code (like `~/tools/pampelmuse`) and check out the trunk of the repository:

```
# svn co http://svn.astro.physik.uni-goettingen.de/pampelmuse/trunk
```

This should download the most recent version of PampelMuse . In order to use the code afterwards, the following modifications should be added to your shell configuration. For a bash shell, use

```
# export pampelmuse_path=<path to PampelMuse>/trunk ## folder were SVN checkout was performed
# export PYTHONPATH=$PYTHONPATH:$pampelmuse_path
# export PATH=$PATH:$pampelmuse_path/pampelmuse/bin
```

and for a c shell use

```
# setenv pampelmuse_path <path to PampelMuse>/trunk ## folder were SVN checkout was performed
# setenv PYTHONPATH $PYTHONPATH:$pampelmuse_path
# setenv PATH $PATH:$pampelmuse_path/pampelmuse/bin
```

Note that the SVN repository also contains the setup script. So you can also execute

```
# python setup.py install
```

and proceed in analogy to Sect. 1.2.1. The downside of this approach is that you will have to reinstall PampelMuse everytime you check out a new version from the repository.

In either case, calling

```
# PampelMuse --help
```

should return no error message.

[8] `http://docs.python.org/2/install/#alternate-installation`

2 The configuration

2.1 Default configuration

The behaviour of PampelMuse can be modified using a number of configuration parameters. The default configuration that is shipped with the code is contained in an XML-file that is stored under `$pampelmuse_path/config/PMparams.xml`. To view this file, open it with any web browser, e.g. via

```
# firefox $pampelmuse_path/config/PMparams.xml .
```

In the following sections, the individual parameters of the configuration will be gradually introduced when discussing the part of PampelMuse that they influence. Note that each parameter is introduced via the statement

NAME type=*type* default=value comment

that provide its name, type (e.g. string or float), default value, and a (hopefully) helpful comment about its usage. Some parameters that influence the overall behaviour of PampelMuse are provided in the following.

VERBOSE type=*integer* default=2	The chattiness of PampelMuse , i.e. the amount of output produced on the console, increasing from 0 to 4. The code uses the logging module of python, the different numbers correspond to the following levels: 0–CRITICAL, 1-ERROR, 2–WARNING, 3–INFO, 4-DEBUG

CPUCOUNT type=*integer* default=1	The number of CPUs used by the parallelized parts of PampelMuse . If set to −1, the code uses all available CPUs on the system.

LAYERRNG type=*tuple* default=(0, −1)	The range of layers of the IFS data that should be analysed. The default corresponds to the full cube.

2.2 Modifying the configuration

I strongly recommend leaving the XML file untouched even if you want to change the configuration. Instead, make use of one of the possibilities introduced in the following to supply your own set of configuration parameters to an analysis.

2.2.1 Using the command line

PampelMuse can be started with the option `--fromline` which accepts a comma-separated list of **name**=*value* pairs, where `value` is the new value of the parameter **name**. As an example, `--fromline verbose=3,ifu=muse,interact=True` would be a valid expression for the `--fromline` argument.

2.2.2 Using a plain text file

Alternatively, PampelMuse may be called with the `--fromfile` option in which case the configuration is updated using a plain text file which contains the **name**=*value* pairs, one per line. As an example, a valid argument would be `--fromfile pampelmuse.user` where the file `pampelmuse.user` may look like this:

```
## Example configuration file
ifu=muse
verbose=3
interact=True
  ...
```

3 The input data

3.1 Supported instruments

In general, PampelMuse is designed to correctly handle IFS data regardless of which instrument was used to observe the data. However, because of the huge diversity of data formats that are produced by the indivdual IF spectrographs, I cannot guarantee that it will work out of the box for each instrument. So far, I successfully tested it using data from

- PMAS (Roth et al., 2005)
- ARGUS (Pasquini et al., 2002)
- MUSE (Bacon et al., 2010)
- KMOS (Sharples et al., 2004)

The routine will try to figure out which instrument the data is coming from by searching for the standard header keyword `INSTRUME`. If that is not present, the following configuration parameter may be used to provide an instrument identifier

IFU	type=*string*	default=pmas	The name of the IFU which has observed the data. Note that this parameter is only used if the code does not find any instrument information in the header of the FITS data. otherwise, preference will be given to the header information.

If you have data from another instrument that is not properly handled by PampelMuse , let me know. On the other hand, if the data is handled correctly, that would be very helpful for me to know as well.

3.2 Preparing the data

First note that PampelMuse is *not* a data reduction pipeline. All analysis steps assume that the IFS data has been properly reduced and that all the instrumental artefacts have been removed. In general, the IFS data can come in two flavours:

1. As a three-dimensional data cube with two spatial axes (`NAXIS1` and `NAXIS2`) and a spectral axis (`NAXIS3`). This format can always be used, even if the data were obtained with an instrument that is unknown to PampelMuse .
2. As a two-dimensional "image" with one spatial and one spectral axis. The advantage of this format is that it can also be used with instruments that have a ireegular (i.e. non-rectangular) spatial sampling. However, for this format to work PampelMuse must know how to arrange to different spaxels in the field of view. Currently this format is only supported for PMAS and ARGUS data.

Besides the actual data, PampelMuse also requires an estimate of the uncertainties or the variances for the analysis. Optionally, a bad pixel mask can be used. Data, uncertainties, and mask can either be provided as different extensions of a single FITS file or as three individual FITS files. In the former case, the routine will use the extension names to identify what is stored in which extension, see Table 1. In the latter case, the individual filenames should have the same prefix and be suffixed `.dat.fits` (the data), `.err.fits` (the uncertainties), and `.msk.fits` (the bad pixel mask).

If provided, the bad pixel mask should be zero for valid pixels and one for pixels that should be excluded. Instead of providing a dedicated mask, one can also set the invalid pixels to NaN and PampelMuse will ignore them in the analysis.

Although not required, it is highly advantageous if the header of the FITS file containing the IFS data contains some basic WCS information, i.e. at least the keywords `RA`, `DEC`, and the matrix `CDi_j`. PampelMuse will use this information to determine the spaxel scale and the approximate location of the field of view on the sky. If no WCS information is available, the keyword `PIXSCALE` should be used to provide the spaxel scale.

Type	required?	EXTNAME
Data	y	DATA
Uncertainties	(y)	ERROR, SIGMA
Variances	(y)	STAT
Bad pixel mask	n	MASK

Table 1: Extension names used to identify the input required for the analysis in a combined FITS file. Note that either uncertainties or variances must be provided.

PIXSCALE	type=*float*	default=1.0	The scale of the spaxels in arcseconds. Note that this parameter is only used if the code cannot infer the spaxel size from the (WCS) information provided in the FITS header of the IFS data.

3.3 The reference catalogue

As PampelMuse does not currently include a dedicated source detection algorithm, the analysis relies on the presence of a reference catalogue, containing coordinates and magnitudes of the stars in and around the observed field. The best sources for such a catalogue are imaging data obtained with the *Hubble* space telescope (HST) or ground based adaptive optics images. Note that the quality of the reference catalogue (especially its completeness and astrometric accuracy) will strongly influence the results of the analysis. For example, a common shortcoming of catalogues generated from HST data is that the brightest stars are often missing because they were saturated in the exposures. If this is the case, try to add them manually.

The reference catalogue should be present as a plain ascii-file with the first (non-commented) line containing the column names. PampelMuse will recognize column names such as `ra`, `dec`, or `id` and process the column content accordingly. The reference catalogue must contain the following columns:

1. A unique identifer for every source; must be a non-negative integer
2. Two columns containing the source coordinates in a reference system. Both pixel and world coordinates are accepted, and priority is given to the latter if both are present. If provided, the world coordinates should be in units of decimal degrees.
3. A magnitude for every source in a passband that covers at least some part of the spectral range of the IFS data. Currently the filter name also needs to be specfied in the configuration, using the parameter

PASSBAND	type=*string*	default=f814w	The name of the filter in which the magnitudes provided in the reference catalogue were observed.

Here is an example of how a valid reference catalogue file may look like:

```
## Example reference catalogue
## Random Cluster NGC9999, generated from HST/WFPC2 data
id    ra    dec   f606w
 1  12.764311  -26.986152  13.29
 2  12.751325  -26.986431  14.87
 4  12.760123  -26.976514  12.25
 ...
```

TBD: Add possibility to include filtercurve if it is missing.

4 The PSF models

PampelMuse currently uses purely analytical PSF profiles. The user can choose between a Gaussian and a Moffat profile. An extensive discussion of the profiles and all associated formulae can be found in Appendix A. Here we restrict the discussion to the main formulae and most important properties. The analytical profile of the Gaussian is

$$G(r) = \Sigma_0 \exp\left\{\frac{-r^2}{2\sigma^2}\right\}, \tag{1}$$

whereas the Moffat profile is described via

$$M(r) = \Sigma_0 \left[1 + \left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta}. \tag{2}$$

The PSF profile that is used in the analysis can be changed using the parameter

PSFTYPE	type=*string*	default=moffat	The name of analytical PSF profile to be used. Currently supported are 'moffat' and 'gaussian'.

For either profile, Σ_0 is the central intensity. The width of the Gaussian is controlled by the standard deviation σ, while the effective radius r_d defines the width of the Moffat. However, instead of σ and β, PampelMuse uses the FWHM as a measure for the profile width (see Appendix A). Note that the FWHM is always measured in units of spaxels.

The Moffat profile has a second shape parameter, β. It measures the kurtosis of the profile, i.e. the strength of the wings of the PSF. Note that higher values of β correspond to less pronounced wings. From my experience, $\beta \sim 2.5$ is a typical value for a seeing-limited PSF. For this value, a Moffat profile has much more pronounced wings than a Gaussian profile. Therefore I strongly recommend using the Moffat profile because the Gaussian profile will almost certainly underestimate the wings of the PSF.

The radius r incorporates the ellipticity e and the position angle θ of each profile. Thus, besides the actual centroid coordinates, the Moffat profile has 4 free parameters, the Gaussian profile has 3. The initial PSF parameters and their behaviour during the analysis are specified using three configuration parameters, `PSFPARS`, `PSFFIT`, and `PSFRAD`.

PSFPARS	type=*tuple*	default=(2.5, 3.0, 0.0, 0.0)	The initial guesses for the PSF parameters. For a Moffat profile, the order is (β, FWHM, e, θ). For a Gaussian profile, the order is (FWHM, e, θ). In either case, the FWHM is measured in units of spaxels.

PSFFIT	type=*integer*	default=6	The PSF behaviour in the fit. PSFFIT = $\sum_n 2^n$, where $n \geq 1$ runs over the indices of the free parameters; e.g. PSFFIT = 6 for a Moffat means to fit β and FWHM at constant e and θ.

PSFRAD	type=*float*	default=10	The centroid distance out to which the profile will be calculated, given in spaxels. Must of course be > 0.

5 The handling of the source coordinates

To obtain IFS spaxel coordinates (x^n, y^n) from the coordinates (u^n, v^n) of source n in the reference catalogue, it is currently assumed that a global transformation exists that can be described by

- a scaling by a factor ξ,
- a rotation around an angle θ,
- shifts along the x- and y-axis of the IFS cube.

In summary, we thus have the following formulae:

$$x^n = \xi \cos\theta\, u^n - \xi \sin\theta\, v^n + C\,, \tag{3}$$
$$y^n = \xi \sin\theta\, u^n + \xi \cos\theta\, v^n + D\,. \tag{4}$$

Substuting $A = \xi \cos\theta$ and $B = -\xi \sin\theta$, we obtain

$$x^n = A\, u^n + B\, v^n + C\,, \tag{5}$$
$$y^n = A\, v^n - B\, u^n + D\,. \tag{6}$$

The parameters A, B, C, and D of the coordinate transformation can be optimized in the analysis. Their behavior during the extraction is controlled via the parameter `POSFIT`.

POSFIT	type=*integer*	default=2	Defines how the coordinate transformation is handled during the analysis: 0 – no optimization; 1 – only optimize the shifts, i.e. parameters A and B; 2 – optimize all 4 parameters of the transformation.

6 The analysis with PampelMuse

PampelMuse provides three main routines that perform the source extraction: INITFIT, CUBEFIT, and POLYFIT. They are described in the following. There are also several auxiliary routines available for various other tasks. To get an overview of all the available routines, type

```
# PampelMuse -h
```

6.1 The PampelMuse GUI

Before describing the individual routines of PampelMuse , the graphical user interface (GUI) that is part of several routines is introduced. The GUI is loaded automatically when running PampelMuse in interactive mode and executing the INITFIT (Sect. 6.2) or the POLYFIT (Sect. 6.4) routine. An example of how the GUI looks like is displayed in Fig. 1.

The GUI shows three separate panels. The upper half is reserved for the spectrum viewer while the lower half contains the reference viewer on the left and the IFS viewer on the right side. The buttons to the top right of each panel allow to zoom in to and out of the data that is displayed, pan over it, and recover previous views of it. The cuts used to display the reference and the IFS data can be changed using the buttons at the bottom of either panel. Finally, the slider to the right of the IFS viewer can be used to scroll through the different layers of the IFS data.

Note that the GUI will only appear if PampelMuse is executed in interactive mode. This is controlled by the parameter `INTERACT`

INTERACT	type=*boolean*	default=True	If True, the code will open a GUI that allows the user to control several steps of the data analysis. Otherwise, these steps are performed without any user interaction. Note that the automated mode requires that the basic WCS information mentioned in Sect. 3 is available in the header of the IFS data.

6.2 INITFIT: Selecting the sources

The first step of the analysis is to identify those sources from the reference catalogue for which it is feasible to extract spectra from the IFS data. As the reference catalogue ideally was obtained from higher resolution data and typically covers a larger area on the sky, it would not make much sense to try and extract a spectrum for each star in the catalogue. Instead, we use INITFIT to make a selection. The syntax for calling this routine is

```
# PampelMuse <config> <IFS data prefix> INITFIT <reference catalogue> .
```

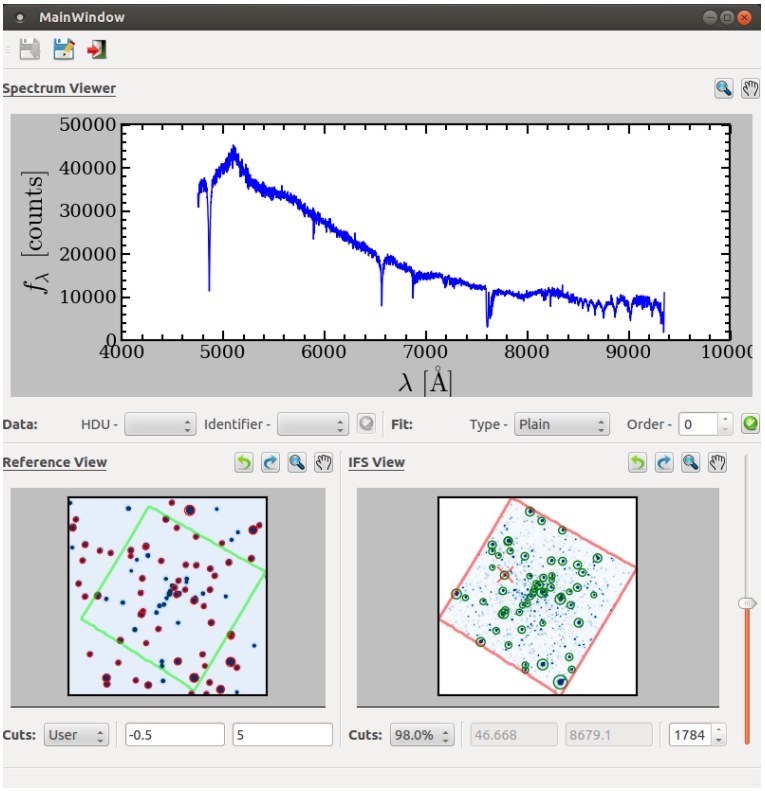


Figure 1: The graphical user interface (GUI) of PampelMuse . The bottom right panel shows a MUSE datacube of NGC 6397 observed during commissioning. The spectrum of the spaxel marked by a red cross is shown in the upper panel. The lower left panel shows a mock image created from a photometric catalogue.

Note that the IFS data should be provided without the `.fits` or `.dat.fits` suffix. The parameter `config` refers to the user setup as explained in detail in Sect. 2.

INITFIT performs the following steps in order to select a useful subset of the available sources.

1. Opening the IFS data, checking their validity.
2. Opening the reference catalogue file, checking its validity.
3. Creating an image from the IFS data in the passband in which the magnitudes in the reference catalogue were provided.
4. Creating a mock image from the sources in the reference catalogue, again in the passband in which the magnitudes were provided. The mock image is prepared using the same pixelsize as the IFS data and the initial guess of the PSF for the IFS data (see parameters `PSFTYPE` and `PSFPARS`, Sect. 4).
   Note: Depending on the coverage of the reference catalogue and on the number of sources in it, this step can take a lot of time and/or consume a lot of memory. If you encounter one of these problems, you can either manually reduce the footprint of the catalogue or set the `RESLIM` parameter (see below) to a smaller magnitude. The latter works because only sources brighter than the assumed confusion limit are used to create the mock image. If at the same time the parameter `UPDATERL` is true (recommended), then this will have no effect on the final source selection.
5. Cross-correlating the two images just created to find the most likely location of the IFS field of view with respect to the reference catalogue. Note that this step requires that both images indeed have the same pixel scale and orientation. To ensure this, RA and Dec coordinates of the sources

must be provided with the reference catalogue and the header of the IFS data must include the WCS information mentioned in Sect. 3. Otherwise this step is skipped.
One issue I encountered when doing the cross-correlation was that a wrong pointing solution was selected because the signal was dominated by the brightest stars in the reference catalogues. To suppress this behaviour, the parameter `CCMAXOFF` can be used.

CCMAXOFF	type=*float*	default=-1.0	The maximum allowed offset with respect to the header WCS position when locating the IFS pointing via cross correlation with the photometric catalog, given in spaxels. The default, −1.0, sets the allowed offset to half the size of the field of view. If set to 0, the pointing suggested by the header WCS will be left unchanged.

6. Loading the PampelMuse GUI with the content following content (also in Fig. 1):

   - The *Reference view* shows the mock image created from the photometric input catalogue. The sources are marked with red circles and are pickable. When selecting a source via the left mouse button, the GUI displays some information about it in the status bar at the bottom of the GUI. If the cross correlation in the previous step was successful, then the reference view also shows the suggested position and orientation of the IFS field of view.
   - The *IFS view* shows a layer from datacube (initially the first one). If the cross-correlation in the previous step was successful, then the GUI will show the suggested positions of the brighter sources as green circles. It is possible to grab individual spaxels from the cube using the left mouse button.
   - The *Spectrum viewer* is initially empty. If a spaxel is grabbed in the IFS view, then the spectrum of that spaxel is displayed.

   Note that this step is skipped if PampelMuse is run in non-interactive mode.

7. From here, there are basically two options on how to proceed:

   (a) If the cross correlation was successful and you are happy with the location of the IFS field of view with respect to the reference catalogue, then you can just close the GUI and let INITFIT proceed with the source selection.

   (b) If the cross correlation failed or you are not happy with its result, then you will have to define the approximate location of the IFS field of view with respect to the reference catalogue manually. To do so, proceed as follows. Pick a source in the reference catalogue, then select its approximate position in the datacube. Once a source is successfully identified, it will be displayed by a filled red circle in the Reference view. To undo the identification for a source, pick it with the *right* mouse button in the Reference view. At least three sources should be identified before closing the GUI. INITFIT will then use this information to re-determine the location of the IFS field of view.

   In non-interactive mode, the code assumes that the location of the IFS field of view determined via cross-correlation is correct.

8. Knowing the positions of the sources in the IFS field of view, INITFIT will then estimate the signal-to-noise ratio (S/N) for all sources from the reference catalogue. To this aim, it assumes a photometric zeropoint in the provided filter, calculates the summed flux of all sources inside the field of view and compares it to the actual flux in the image created from the IFS data (cf. step 3). The photometric zeropoint is then adapted to bring the two fluxes in agreement. If instead a fixed photometric zeropoint should be used (not recommended), the parameter `ZEROMAG` may be used.

ZEROMAG	type=*float*	default=99.9	Sets the photometric zeropoint of the IFS data in the provided passband, i.e. the magnitude of a source creating 1 count/layer. If set to 99.9 (recommended), the zeropoint is estimated from the data.

Depending on the size of the reference catalogue, this step may take some time, even though it is parallelized.

9. The next step is the actual source selection. INITFIT will loop over the sources in the reference catalogue – sorted by magnitude, i.e. brightest first – and do a number of checks to determine the status of every source in the fitting process:

   - Check if source is brighter than the fixed confusion limit `RESLIM`. This is only done if `UPDATERL` is False (not recommended).

RESLIM	type=*float*	default=16.0	The resolution limit, i.e. the magnitude below which sources cannot be resolved anymore. Note that this limit is automatically determined if UPDATERL is set to True.

UPDATERL	type=*boolean*	default=True	If True, the resolution limit is automatically determined based on the local source density (see parameter CONFLIM). Otherwise, the fixed value provided by RESLIM is used.

   - Check if the local source density is below the provided threshold, `CONFDENS`. As for a larger field of view, such as the MUSE WFM, the stellar density can be very different across its extent, the confusion limit is estimated locally in an area of radius $10 \times$ FWHM around each star.

CONFDENS	type=*float*	default=0.4	The confusion density, i.e. the maximum density of resolvable sources per resolution element. IF UPDATERL is True, the local resolution limit will be the magnitude at which the density of brighter sources reaches this density. The default value of 0.4 was determined using simulations (see Kamann et al., 2013).

   - Check if S/N is above the threshold value, `SNRCUT`.

SNRCUT	type=*float*	default=3.0	The minimum signal to noise of sources that are included in the extraction.

     Note that the S/N estimated in the previous step is modified in two cases.

     (a) If a source is outside the IFS field of view, the S/N is penalized by $\exp(-(d/\mathrm{FWHM})^2)$, where d is the distance to the nearest spaxel.

     (b) If a source is nearby a brighter source (with a distance $r <$ FWHM), the S/N is modified with the factor $1. + 0.9 \log(r/\mathrm{FWHM})$. This factor approximates the decrease in S/N found for nearby sources in Kamann et al. (2013).

   - Check that there is no brighter source in the direct vicinity, defined by the parameter `BINRAD`, of the source.

BINRAD	type=*float*	default=0.3	The minimum spatial separation (in units of the PSF FWHM) of a source that is considered resolvable to a brighter source.

   Sources that passed all the above checks are assigned status 2. They are considered resolved and their spectra are set for extraction. Sources that fail `BINRAD` criterion or the S/N criterion because of the vicinity of a brighter source are given status 1 and their contribution is accounted for in the extraction of the spectrum of the brighter neighbour. All other stars are given status -1 and are ignored during the spectrum extraction.

10. The next step is only carried out if an unresolved stellar component should be included in the extraction process. The idea here is that the sources that are fainter than the resolution limit still create a grainy structure across the field of view (similar to surface brightness fluctuations in

nearby galaxies). If the reference catalogue is of sufficient depth, this graininess can be modelled using the broadband magnitudes, positions, and the PSF. In the extraction, a combined spectrum will be extracted for the entity of these sources. If such a component is used and which sources enter it is defined by the parameters `USEUNRES` and `UNRESBIN`.

USEUNRES	type=*boolean*	default=False	If True, the routine will collect set all sources inside the IFS field of view that are less than UNRESBIN magnitudes fainter than the faintest resolved source and extract a combined spectrum for them.

UNRESBIN	type=*float*	default=2.0	The width of the magnitude interval in the sense that the faintest sources included in the unresolved component are UNRESBIN magnitudes fainter than the faintest resolved sources. Only used if USEUNRES is True.

All sources in the unresolved component are given status 0.

11. To account for the spectrum of the night sky, an additional component that is spatially flat may be included in the extraction.

GLOBSKY	type=*boolean*	default=True	If True, an additional component will be included in the extraction that accounts for the night sky (or a stellar component that is spatially flat).

12. The next step depends on the size of the IFS data, more precisely on the number of spaxels. PampelMuse was designed to handle data coming from various instruments, in particular PMAS and MUSE. Now, PMAS has 256 spaxels while MUSE has more than 90 000. As a consequence, the way the PSF is optimized for the two instruments is different. Within a field of view as small as the one of PMAS, you will almost never find reasonably isolated and bright stars that allow you to constrain the PSF. Instead, for PMAS-like data the idea is to fit a PSF profile to all resolved sources simultaneously. While this approach works well for ~ 20 sources which you typically resolve in a PMAS cube, it is computationally very demanding for the $\sim 5\,000$ sources you expect in a MUSE cube. However, for MUSE data the chances of finding good PSF stars is much higher. To account for this diversity in IFS data, PampelMuse uses the parameter `SMALLIFU`.

SMALLIFU	type=*bool*	default=True	This flag defines whether all resolved sources are used to fit the PSF and the coordinate transformation (if True) or only a subset of reasonably bright and isolated PSF sources.

My recommendation is to set `SMALLIFU` to True for cubes with less than $\sim 2\,000$ spaxels and to False otherwise. In the latter case, INITFIT will search for suitable sources to model the PSF during the extraction. Potential PSF sources are those that fulfil the following criteria:

- at least 3 magnitudes brighter than measured resolution limit
- identified as resolved (i.e. status 2) in the source selection
- an estimated S/N > 20
- a distance $\geq$ FWHM from the edge of the IFS field of view
- two isolation criteria:
  - The relative contribution from neighbouring sources inside the region where the PSF is modelled (`PSFRAD`) must be smaller than the value of parameter `APERNOIS`.

EXTNAME	Type	Purpose
PRIMARY	Primary	Header contains the configuration of PampelMuse
SOURCES	Binary Table	The source catalogue, contains basic information about every source used in the extraction.
SPECTRA	Image	Contains the extracted spectra and an estimate of the uncertainties.
POSPARS	Binary Table	Contains the coordinate transformation from reference to IFS coordinates.
POSITIONS	Binary Table	Contains the parameters of the polynomial fits to the IFS coordinates.
PSFPARS	Binary Table	Contains the parameters of the fitted PSF

Table 2: The different extensions of the PRM-file and their respective contents.

APERNOIS	type=*float*	default=0.1	The maximum allowed contamination around a PSF source from a neighbouring source. It is measured as the flux ratio of the brightest source within a distance PSFRAD and the flux of the PSF star itself.

– No brighter sources are allowed inside 1.5× PSFRAD distance.

Aperture photometry is performed on each layer for each source that fulfils the above criteria. The aperture radius is defined by the configuration parameter APERRAD.

APERRAD	type=*float*	default=3.0	The radius in spaxels of the circle around each PSF source in which the aperture photometry is performed.

Note that not only the fluxes are measured, but also the first and second order spatial moments. After that, the resulting spectra are used to redetermine the S/N estimate of each source and to perform the final selection of PSF sources. Also, the centroids of the aperture photometry are used to estimate the source coordinates as a function of wavelength. Finally, the second order moments measured for the PSF sources are used to estimate the ellipticity and the position angle of the PSF (if they are free parameters in the PSF fit, see Sect. 4).

13. Write the results from the source selection to a dedicated multi-extension FITS-file, named *<prefix>.prm.fits* and hereafter called PRM-file. This file is loaded by all other routines of PampelMuse . An overview on the structure of the PRM-file is given in Table 2.

6.3 CUBEFIT - Extracting the spectra

CUBEFIT is the routine that actually performs the extraction of the spectra by simultaneously fitting a PSF to all sources that have been identified as resolvable by INITFIT. The calling sequence for CUBEFIT is

```
# PampelMuse <config> <IFS data prefix> CUBEFIT .
```

Note that the routine expects a file PRM-file generated by INITFIT. This file also contains the configuration parameters that were provided to INITFIT in the header of the primary HDU. If a dedicated configuration is provided to CUBEFIT (or any other routine that requires the PRM-file), preference is given to the parameter values in the provided configuration. However, only a subset of the configuration parameters can be changed after running INITFIT. In general, all parameters that influence the source selection (such as RESLIM, PSFPARS, ...) cannot be changed any more.

CUBEFIT analyses the IFS data layer by layer, starting from the central layer and simulateously proceeding into the red and the blue part of the cube. On each layer, the code performs an interative analysis in order to extract the optimal flux for each source. This iterative scheme, also depicted in Fig. 2, can be summarized as follows.

1. Using the initial guesses for the PSF and the coordinate transformation, a first fit of the fluxes is performed. The flux fit is a linear least-squares fit that can be solved by direct matrix inversion and does not require initial guesses. The matrix contains the PSF profiles of the individual components. As each PSF profile is significantly different from zero only in the vicinity of the represented star, the majority of the matrix entries are zero. For such sparse matrices very efficient inversion algorithms are available. CUBEFIT uses the method `lsmr` (Fong and Saunders, 2010) from the `scipy.sparse.linalg` package. It was tested for up to 10 000 sources and provided reliable solutions in reasonable computation times.

2. With an initial value for all fluxes available, CUBEFIT updates the coordinate transformation from reference to IFS coordinates via the Levenberg-Marquardt algorithm `leastsq` from the `scipy.optimize` package. To speed up the computation, CUBEFIT internally calculates the Jacobi matrix, i.e. the matrix containing the partial derivatives of the PSF profile(s) with respect to the fitted parameters of the coordinate transformation, also provided in App. A.

3. The third step, the optimization of the PSF parameters, uses the fluxes and the updated coordinate transformation. Again, the code makes use of the `leastsq` algorithm and incorporates the Jacobi matrix, this time containing the partial derivatives with respect to the PSF shape parameters.

4. The updated PSF and coordinate transformation are used to refit the fluxes of the sources. Afterwards, it is checked whether the fluxes converged. The convergence criterion that is used by PampelMuse is that the median change in flux compared to the last iteration is < 0.005. If the convergence criterion is fulfilled or the maximum number of iterations has been reached, the analysis is continued with the next layer. Otherwise the iteration continues with fitting the coordinate transformation parameters (step 2). The maximum number of iterations can be set by the parameter `MAXITER`.

MAXITER	type=*integer*	default=10	The maximum number of iterations that is performed by CUBEFIT on an individual layer of the IFS data.

5. After all the requested layers have been analysed in this way, the results are written to the PRM-file.

Note that the initial after the first two layers have been processed, the code will use the fit results for the PSF and the coordinate transformation as initial guesses for the analysis of the next layer. To avoid that a failed fit in one layer (e.g. because of a strong cosmic) influences the analysis of the following ones, the initial guesses are obtained by averaging over the results from the 10 adjacent layers.

It should be highlighted again that the way the fit of the coordinate transformation and PSF parameters is carried out depends on the value of the parameter `SMALLIFU`, cf. Sect. 6.2. If the value of `SMALLIFU` is True, the code optimizes for the parameters by fitting a PSF profile to all resolved sources simulateously. Otherwise, the code loops over all PSF sources found by INITFIT and fits a single PSF profile to them. Afterwards, the parameters of the coordinate transformation are obtained using the reference coordinates and the fitted centroids of all PSF sources. The PSF parameters are updated by averaging over the results from the individual PSF fits. To remove contamination from neighbouring sources as accurately as possible, those sources are subtracted from the IFS data before the fit using the current PSF estimate.

Note that this routine is computationally the most demanding part of PampelMuse . The executions times can vary from a few minutes if only some sources are present to tens of hours for really crowded fields and huge datasets. My recommendation is to start CUBEFIT in a `screen` session, detach from it and have a good glass of wine. (Note however that for budget reasons, the wine is not shipped with PampelMuse .)

6.4 POLYFIT

After running CUBEFIT, you can check the results using the PampelMuse GUI. To this aim, start the POLYFIT routine via

`# PampelMuse <config> <IFS data prefix> POLYFIT`. This routine serves two purposes. On the one hand, it allows for a visual inspection of all fitted parameters, i.e. the extracted spectra, the IFS

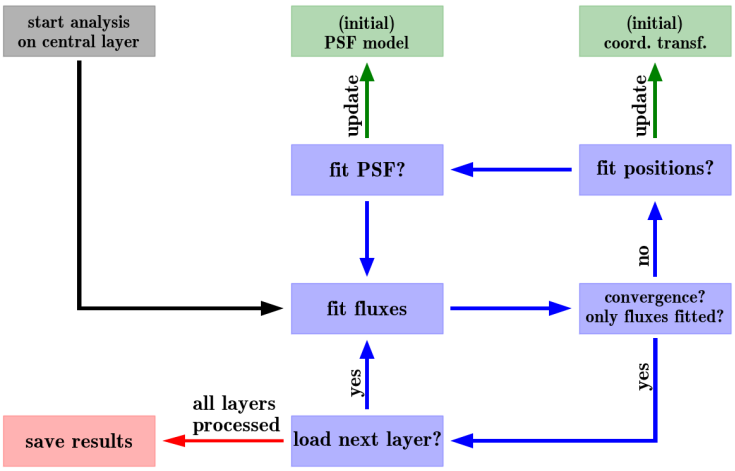


Figure 2: The iterative fittting approach of the CUBEFIT routine.

coordinates, and the PSF. On the other hand, it can be used to fit wavelength dependencies of the PSF parameters and the IFS coordinates with polynomials. The idea behind the latter is that from a theoretical point of view, you expect those quantities to vary smoothly with wavelength, e.g. because of differential atmospheric refraction (DAR). But because of the limited S/N in each layer, the fitted variation with wavelength will show scatter, especially in spectral regions were the signal is low like telluric absorption bands. Fitting those variations with polynomials averages out the scatter and should in principle lead to a higher S/N in the extracted spectra. When starting CUBEFIT again after performing the polynomial fits, the routine will only refit the fluxes in each layer while keeping the other parameters fixed at their polynomial values.

When running PampelMuse in interactive mode, the routine will load the GUI with the content that is displayed in Fig. 3. In large parts it looks similar to the one that was opened by INITFIT. In the following the functionality of the GUI is explained.

- The purpose of the Spectrum Viewer is to display the wavelength dependecies of the fitted parameters of spectra. The data to be shown can be selected using the two drop-down menus in 'Data:' to the bottom left of the panel. Via 'HDU', you can select the parameter class (spectrum, coordinate, PSF parameter) to be displayed, while 'Identifier' lets you select the ID in case that a spectrum or coordinate should be displayed or the specific parameter in case the PSF or the coordinate transformation should be displayed. The requested data is displayed upon clicking the green button.
  Using the left mouse button, you can exclude wavelength ranges from the polynomial fits. To this aim, keep the mouse button pressed while scanning over the desired range. Doing the same with the right mouse button lets you unmask regions.
  The two drop-down menus in 'Fit:' to the bottom right of the panel allow you to specify the type and order of the polynomial that is used to fit the data. The fit is carried out via a click on the green button to the right. Note that currently only fits to the PSF parameters and the IFS coordinates are supported. Fitting the individual IFS coordinates instead of the coordinate transformation (which was used to calculate them) is preferred because the parameters of the coordinate transformation can sometimes show correlated variations (in particular if the field of view contains only a few spaxels) that are difficult to describe with polynomials.

- The IFS View displays again the cube, this time with the positions of the extracted sources overplotted. The size of the circles used to mark the extracted sources sclae with their brightness. Sources marked by green circles are used to fit the PSF. While this display is very helpful for small fields of view, it can look very messy for MUSE-like data of crowded stellar fields. However, you can use the magnifying glass to the upper right of the panel to zoom into the data. When clicking on a source while the Spectrum View shows a spectrum or an IFS coordinate, the spectrum/coordinate of the selected source will be shown.

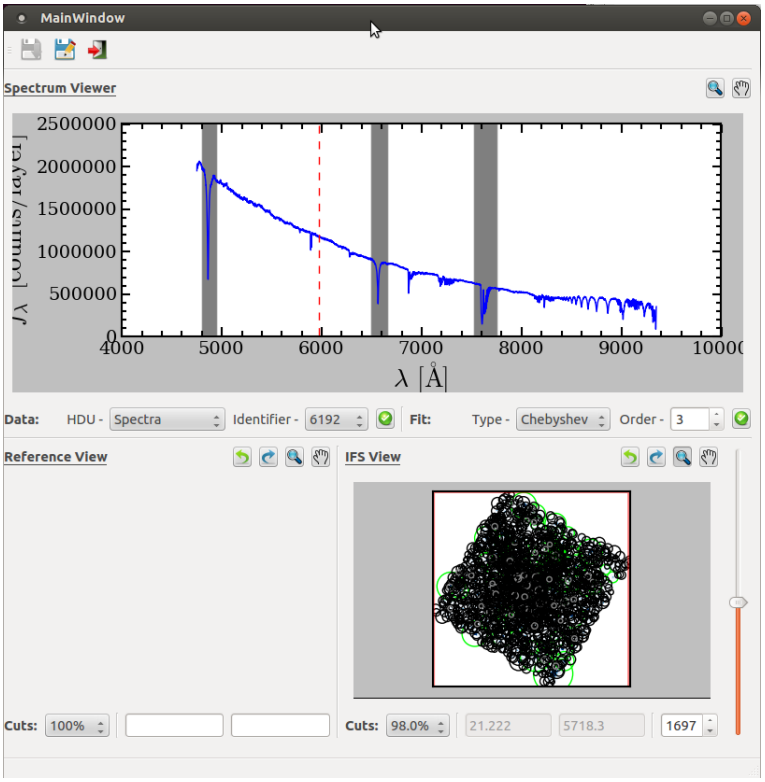


Figure 3: The GUI that is loaded by the POLYFIT routine when using PampelMuse in interactive mode.

- The Reference View is currently not used.

After you have fitted all the desired parameters and are happy with the results, you can use the buttons to the top left of the GUI to save the results to the PRM-file.

Note that when PampelMuse is not run in interactive mode (see parameter `INTERACT`), POLYFIT will not show the GUI but use polynomials of requested order to fit the source coordinates and the free PSF parameters automatically. The type and order of the polynomials being used can be controlled using the configuration parameters `POLTYPE` and `POLYDEG`.

POLYTYPE	type=*string*	default=plain	The type of polynomial used to fit the wavelength dependencies. Can be one of 'plain' (normal polynomial), 'chebyshev', 'legendre', and 'hermite'.

POLYDEG	type=*integer*	default=0	The degree of the polynomials used to fit the wavelength dependencies.

In order to be able to mask out wavelength ranges also when running POLYFIT in non-interactive mode, it can be called with the option `-mask <wmin> <wmax>`, indicating that the wavelength range between `<wmin>` and `<wmax>` should be masked out. This option may be provided more than once so that multiple ranges can be excluded.

6.5 GETSPECTRA - obtaining the results

Once the analysis is complete, you probably want to have the spectra in some other format than the PRM-file. To facilitate this, PampelMuse has a routine GETSPECTRA, called with the following sequence

`# PampelMuse <config> <IFS data prefix> GETSPECTRA <spectra prefix>`. The routine will store all the extracted spectra in individual FITS files that contain the spectrum in the primary extension and an

estimate of the uncertainties in an image HDU named `SIGMA`. The header of each file will contain the correct dispersion information, general information about the star and the spectrum as well as a number of hierarch keywords copied from the PRM-file. Note that this routine is called with the mandatory argument `<spectra prefix>` which is a string (typically the name of the observed cluster, galaxy, ...) that is used as a common prefix to all FITS files that are written. Additionally, the routine may be called with the `-folder` option to provide a folder in which all spectra should be stored.

6.6 Recommended analysis strategy

The strategy I usually use when analysing IFS data is the following. After selecting the sources for which spectra should be extracted via INITFIT, I perform a first extraction with CUBEFIT in which the PSF parameters and the coordinate transformation are optimized for each layer. Afterwards, I use POLYFIT to smooth the wavelength dependencies of those parameters via polynomial fits. Finally, I run CUBEFIT again. Since the PRM-file now contains the polynomial fits to the positions and PSF parameters, it will only fit the source fluxes without any iteration. Hence, the secoond run of CUBEFIT should be much faster than the first.

References

R. Bacon, M. Accardo, L. Adjali, H. Anwand, S. Bauer, I. Biswas, J. Blaizot, D. Boudon, S. Brau-Nogue, J. Brinchmann, P. Caillier, L. Capoani, C. M. Carollo, T. Contini, P. Couderc, E. Daguisé, S. Deiries, B. Delabre, S. Dreizler, J. Dubois, M. Dupieux, C. Dupuy, E. Emsellem, T. Fechner, A. Fleischmann, M. François, G. Gallou, T. Gharsa, A. Glindemann, D. Gojak, B. Guiderdoni, G. Hansali, T. Hahn, A. Jarno, A. Kelz, C. Koehler, J. Kosmalski, F. Laurent, M. Le Floch, S. J. Lilly, J.-L. Lizon, M. Loupias, A. Manescau, C. Monstein, H. Nicklas, J.-C. Olaya, L. Pares, L. Pasquini, A. Pécontal-Rousset, R. Pelló, C. Petit, E. Popow, R. Reiss, A. Remillieux, E. Renault, M. Roth, G. Rupprecht, D. Serre, J. Schaye, G. Soucail, M. Steinmetz, O. Streicher, R. Stuik, H. Valentin, J. Vernet, P. Weilbacher, L. Wisotzki, and N. Yerle. The MUSE second-generation VLT instrument. In *Society of Photo-Optical Instrumentation Engineers (SPIE) Conference Series*, volume 7735 of *Society of Photo-Optical Instrumentation Engineers (SPIE) Conference Series*, July 2010. doi: 10.1117/12.856027.

D. Fong and M. Saunders. LSMR: An iterative algorithm for sparse least-squares problems. *ArXiv e-prints*, June 2010.

S. Kamann, L. Wisotzki, and M. M. Roth. Resolving stellar populations with crowded field 3D spectroscopy. *A&A*, 549:A71, January 2013. doi: 10.1051/0004-6361/201220476.

L. Pasquini, G. Avila, A. Blecha, C. Cacciari, V. Cayatte, M. Colless, F. Damiani, R. de Propris, H. Dekker, P. di Marcantonio, T. Farrell, P. Gillingham, I. Guinouard, F. Hammer, A. Kaufer, V. Hill, M. Marteaud, A. Modigliani, G. Mulas, P. North, D. Popovic, E. Rossetti, F. Royer, P. Santin, R. Schmutzer, G. Simond, P. Vola, L. Waller, and M. Zoccali. Installation and commissioning of FLAMES, the VLT Multifibre Facility. *The Messenger*, 110:1–9, December 2002.

M. M. Roth, A. Kelz, T. Fechner, T. Hahn, S.-M. Bauer, T. Becker, P. Böhm, L. Christensen, F. Dionies, J. Paschke, E. Popow, D. Wolter, J. Schmoll, U. Laux, and W. Altmann. PMAS: The Potsdam Multi-Aperture Spectrophotometer. I. Design, Manufacture, and Performance. *PASP*, 117:620–642, June 2005. doi: 10.1086/429877.

R. M. Sharples, R. Bender, M. D. Lehnert, S. K. Ramsay Howat, M. N. Bremer, R. L. Davies, R. Genzel, R. Hofmann, R. J. Ivison, R. Saglia, and N. A. Thatte. KMOS: an infrared multiple-object integral field spectrograph for the ESO VLT. In A. F. M. Moorwood and M. Iye, editors, *Ground-based Instrumentation for Astronomy*, volume 5492 of *Society of Photo-Optical Instrumentation Engineers (SPIE) Conference Series*, pages 1179–1186, September 2004. doi: 10.1117/12.550495.

A The PSF profiles

An overview of the functional forms of the analytical PSF profiles that PampelMuse uses. Besides the functional forms, we also provide the partial derivatives with respect to the individual profile parameters. They are used to determine the Jacobi matrix in the Levenberg-Marquard algorithm that performs the non-linear least squares fit.

A.1 Handling of positions

All profiles are defined as a function of r, with r giving the distance of a pixel $(x_i,\, y_i)$ to the centroid. In the most general case, the PSF will be of elliptical shape, with its semi-major rotated by an angle θ **with respect to the y-axis**. For this reason, we define a coordinate system $(\hat{x},\, \hat{y})$, that has the same origin as the PSF profile and has the $\hat{x}$-axis aligned with the semi-major axis. Assuming the central coordinates of profile n ($n \in [0, N)$) in the original system are $(x^n,\, y^n)$, the transformation is defined as

$$\hat{x} = (x - x^n)\sin\theta - (y - y^n)\cos\theta\,, \tag{7}$$

$$\hat{y} = (x - x^n)\cos\theta + (y - y^n)\sin\theta\,. \tag{8}$$

The radius r is measured in units of pixels in direction of the semi-major axis of the profile, that is, for an arbitrary position angle θ with respect to that axis, r is calculated using

$$r(x, y) = \sqrt{\hat{x}^2 + \left(\frac{\hat{y}}{1-e}\right)^2}\,, \tag{9}$$

where e is the ellipticity of the profile.

The above equations imply that r is a function of $2N + 2$ parameters that can be free fit parameters in the PSF optimization: x^n, y^n, θ, and e. The partial derivatives of r with respect to those parameters are

$$\begin{aligned}\partial_{x^n} r \equiv \frac{\partial r}{\partial x^n} &= \left[\hat{x}^2 + \left(\frac{\hat{y}}{1-e}\right)^2\right]^{-1/2} \frac{1}{2}\left[2\hat{x}\frac{\partial \hat{x}}{\partial x^n} + \frac{2\hat{y}}{(1-e)^2}\frac{\partial \hat{y}}{\partial x^n}\right] \\ &= -\frac{1}{r}\left[\hat{x}\sin\theta + \frac{\hat{y}\cos\theta}{(1-e)^2}\right]\,, \end{aligned} \tag{10}$$

$$\begin{aligned}\partial_{y^n} r \equiv \frac{\partial r}{\partial y^n} &= \left[\hat{x}^2 + \left(\frac{\hat{y}}{1-e}\right)^2\right]^{-1/2} \frac{1}{2}\left[2\hat{x}\frac{\partial \hat{x}}{\partial y^n} + \frac{2\hat{y}}{(1-e)^2}\frac{\partial \hat{y}}{\partial y^n}\right] \\ &= \frac{1}{r}\left[\hat{x}\cos\theta - \frac{\hat{y}\sin\theta}{(1-e)^2}\right]\,, \end{aligned} \tag{11}$$

$$\begin{aligned}\partial_{\theta} r \equiv \frac{\partial r}{\partial \theta} &= \left[\hat{x}^2 + \left(\frac{\hat{y}}{1-e}\right)^2\right]^{-1/2} \frac{1}{2}\left[2\hat{x}\frac{\partial \hat{x}}{\partial \theta} + \frac{2\hat{y}}{(1-e)^2}\frac{\partial \hat{y}}{\partial \theta}\right] \\ &= \frac{\hat{x}\hat{y}}{r}\left[1 - \frac{1}{(1-e)^2}\right]\,, \end{aligned} \tag{12}$$

$$\begin{aligned}\partial_{e} r \equiv \frac{\partial r}{\partial e} &= \left[\hat{x}^2 + \left(\frac{\hat{y}}{1-e}\right)^2\right]^{-1/2} \frac{1}{2}\cdot\frac{2\hat{y}^2}{(1-e)^3}\,. \\ &= \frac{\hat{y}^2}{r\cdot(1-e)^3} \end{aligned} \tag{13}$$

However, the usual approach in PampelMuse is that we do not fit source coordinates directly, but that we assume they are already known in some reference frame and the task is to optimize the coordinate transformation from the reference system into the actual system of the datacube. Starting from the reference coordinates $(u,\, v)$, we obtain the actual coordinates via

$$x^n = A\,u^n + B\,v^n + C\,, \tag{14}$$

$$y^n = A\,v^n - B\,u^n + D\,. \tag{15}$$

In this case, r is a function of 6 potentially free parameters: A, B, C, D, θ, and e. Accordingly, the partial derivatives $\partial_{x^n} r$ and $\partial_{y_n} r$ must be replaced by the partial derivatives with respect to A–D,

$$\begin{aligned}
\partial_A r \equiv \frac{\partial r}{\partial A} =& \partial_{x^n} r \cdot \frac{\partial x^n}{\partial A} + \partial_{y^n} r \cdot \frac{\partial y^n}{\partial A} = \partial_{x^n} r \cdot u^n + \partial_{y^n} r \cdot v^n \\
=& \frac{1}{r}\left[\hat{x} \cdot (v^n \cos\theta - u^n \sin\theta) - \frac{\hat{y} \cdot (u^n \cos\theta + v^n \sin\theta)}{(1-e)^2}\right], \qquad (16) \\
\partial_B r \equiv \frac{\partial r}{\partial B} =& \partial_{x^n} r \cdot \frac{\partial x^n}{\partial B} + \partial_{y^n} r \cdot \frac{\partial y^n}{\partial B} = \partial_{x^n} r \cdot v^n - \partial_{y^n} r \cdot u^n \\
=& -\frac{1}{r}\left[\hat{x} \cdot (u^n \cos\theta + v^n \sin\theta) + \frac{\hat{y} \cdot (v^n \cos\theta - u^n \sin\theta)}{(1-e)^2}\right], \qquad (17) \\
\partial_C r \equiv \frac{\partial r}{\partial C} =& \partial_{x^n} r \cdot \frac{\partial x^n}{\partial C} = \partial_{x^n} r\,, \qquad (18) \\
\partial_D r \equiv \frac{\partial r}{\partial D} =& \partial_{y^n} r \cdot \frac{\partial y^n}{\partial D} = \partial_{y^n} r\,. \qquad (19)
\end{aligned}$$

A.2 The Gaussian profile

The functional form of the Gaussian profile as it is used in PampelMuse is

$$G(r, e, \sigma, I) = \frac{I}{2\pi\sigma^2(1-e)} \exp\left\{\frac{-r^2}{2\sigma^2}\right\}. \qquad (20)$$

Note, however, that instead of the standard deviation σ, we use the FWHM to control the width of the Gaussian profile. The conversion between the two quantities is given as

$$\mathrm{FWHM} = \sqrt{8 \cdot \ln 2} \cdot \sigma\,. \qquad (21)$$

Furthermore, we control the integrated intensity I of the Gaussian (and any other) profile by providing a magnitude, i.e.,

$$I = 10^{-0.4 \cdot m}. \qquad (22)$$

The partial derivatives with respect to the individual parameters that define the shape of the profile (FWHM, σ, and θ) are calculated using the following formulae:

$$\begin{aligned}
\partial_{\mathrm{FWHM}} G &= \partial_\sigma G / \sqrt{8 \cdot \ln 2} \\
&= \exp\left\{\frac{-r^2}{2\sigma^2}\right\}\left(\frac{-I}{\pi\sigma^3(1-e)} + \frac{I \cdot r^2}{2\pi\sigma^5(1-e)}\right)/\sqrt{8 \cdot \ln 2} \\
&= G\left(\frac{r^2}{\sigma^3} - \frac{2}{\sigma}\right)/\sqrt{8 \cdot \ln 2}\,, \qquad (23) \\
\partial_e G &= \exp\left\{\frac{-r^2}{2\sigma^2}\right\}\left(\frac{I}{2\pi\sigma^2(1-e)^2} - \frac{I \cdot r \cdot \partial_e r}{2\pi\sigma^4(1-e)}\right) \\
&= G\left(\frac{1}{1-e} - \frac{r \cdot \partial_e r}{\sigma^2}\right), \qquad (24) \\
\partial_\theta G &= \frac{I}{2\pi\sigma^2(1-e)} \exp\left\{\frac{-r^2}{2\sigma^2}\right\} \cdot \frac{-r \cdot \partial_\theta r}{\sigma^2} \\
&= -G \cdot \frac{r \cdot \partial_\theta r}{\sigma^2}\,, \qquad (25)
\end{aligned}$$

where $\partial_e r$ and $\partial_\theta r$ are given as defined in Sect. A.1.

Regarding the partial derivatives with respect to any parameter connected to the centroid of the profile, the corresponding formulae given in Sect. A.1 are modified by a common factor,

$$\begin{aligned}
\partial_\nu G &= \frac{I}{2\pi\sigma^2(1-e)} \exp\left\{\frac{-r^2}{2\sigma^2}\right\} \cdot \frac{-r \cdot \partial_\nu r}{\sigma^2} \\
&= -G \cdot \frac{r \cdot \partial_\nu r}{\sigma^2}\,, \qquad (26)
\end{aligned}$$

with $\nu \in \{x^n, y^n, A, B, C, D\}$.

A.3 The Moffat profile

The functional form of the Moffat profile that is included in PampelMuse is given as

$$M(r, e, r_\mathrm{d}, \beta, I) = \frac{I \cdot (\beta - 1)}{\pi r_\mathrm{d}^2 \cdot (1 - e)} \left[1 + \left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta} , \tag{27}$$

where the width of the profile is mainly defined by the effective radius r_d and β controls the kurtosis of the profile. The integrated intensity I is the same as defined in Eq. 22. Similar to our approach for the Gaussian profile, we rather work with the FWHM instead of the effective radius r_d. The FWHM of the Moffat profile can be calculated via

$$\mathrm{FWHM} = 2\sqrt{2^{1/\beta} - 1} \cdot r_\mathrm{d} \,. \tag{28}$$

Equation 28 implies that the FWHM is a function of both, r_d and β. It should not be mixed up with the FWHM of a Gaussian profile that is a measure for the seeing during the observations. In general, the FWHM of a Moffat profile will be smaller than the seeing. This can be verified from Fig. 4, where we have plotted the FWHM ratio between a Gaussian and a Moffat profile as a function of β. The plotted ratios were obtained by fitting a Gaussian profile to the flux distribution created by a Moffat profile with given FWHM and β. Figure 4 shows that the shape of the Moffat profile approaches the shape of the Gaussian profile as β increases. Larger β values correspond to profiles with less pronounced wings. However, the Moffat profile will always have more pronounced wings than the Gaussian.

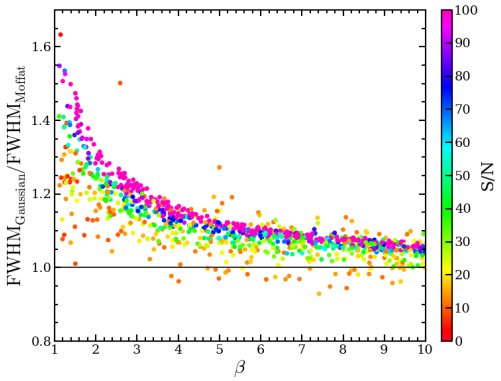


Figure 4: Inverse ratio between the FWHM of a Moffat profile and the FWHM of its best Gaussian fit, as a function of the β value of the Moffat profile and its S/N.

When calculating the partial derivatives of Eq. 27, we have to keep in mind that $r_\mathrm{d} = r_\mathrm{d}(\beta, \mathrm{FWHM})$, and therefore

$$\begin{aligned} \partial_\beta r_\mathrm{d} &= \frac{\mathrm{FWHM}}{2} \cdot \frac{-1}{(2^{1/\beta} - 1)^{-3/2}} \cdot \frac{-2^{1/\beta} \cdot \ln 2}{\beta^2} \\ &= \frac{r_\mathrm{d} \cdot \ln 2}{2 \cdot \beta^2 \cdot (1 - 2^{1/\beta})} \end{aligned} \tag{29}$$

$$\partial_\mathrm{FWHM} r_\mathrm{d} = \frac{1}{2\sqrt{2^{1/\beta} - 1}} \,. \tag{30}$$

With respect to the shape parameters FWHM, β, θ and e we thus obtain the partial derivatives

$$\begin{aligned}
\partial_{\mathrm{FWHM}} M &= \frac{-2I\cdot(\beta-1)}{\pi r_\mathrm{d}^3\cdot(1-e)}\cdot\partial_{\mathrm{FWHM}} r_\mathrm{d}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta} \\
&\quad - \frac{I\cdot\beta\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta-1}\frac{-2r^2}{r_\mathrm{d}^3}\partial_{\mathrm{FWHM}} r_\mathrm{d} \\
&= \frac{2M}{\mathrm{FWHM}}\left[\beta\left(\frac{r}{r_\mathrm{d}}\right)^2\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-1} - 1\right], \qquad (31)\\
\partial_\beta M &= \left(\frac{I}{\pi r_\mathrm{d}^2\cdot(1-e)} - \frac{2I\cdot(\beta-1)}{\pi r_\mathrm{d}^3\cdot(1-e)}\cdot\partial_\beta r_\mathrm{d}\right)\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta} \\
&\quad - \frac{I\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta}\left(\ln\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right] - \frac{2\beta r^2}{r_\mathrm{d}^3\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]}\partial_\beta r_\mathrm{d}\right) \\
&= M\left(\frac{1}{\beta-1} - \frac{2\partial_\beta r_\mathrm{d}}{r_\mathrm{d}} - \ln\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right] + \frac{2\beta r^2}{r_\mathrm{d}^3\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]}\partial_\beta r_\mathrm{d}\right) \\
&= M\left(\frac{1}{\beta-1} - \ln\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right] - \frac{2\partial_\beta r_\mathrm{d}}{r_\mathrm{d}}\cdot\left[1-\frac{\beta\cdot\left(\frac{r}{r_\mathrm{d}}\right)^2}{1+\left(\frac{r}{r_\mathrm{d}}\right)^2}\right]\right), \qquad (32)\\
\partial_\theta M &= \frac{-\beta\cdot I\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta-1}\frac{2r}{r_\mathrm{d}^2}\cdot\partial_\theta r \\
&= -\frac{2M\cdot\beta\cdot r}{r_\mathrm{d}^2}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-1}\cdot\partial_\theta r\,, \qquad (33)\\
\partial_e M &= \frac{I\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)^2}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta} \\
&\quad - \frac{\beta\cdot I\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta-1}\frac{2r}{r_\mathrm{d}^2}\cdot\partial_e r \\
&= M\left(\frac{1}{1-e} - \frac{2\beta\cdot r\cdot\partial_e r}{r_\mathrm{d}^2}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-1}\right). \qquad (34)
\end{aligned}$$

Just like in the case of the Gaussian profile, the partial derivatives with respect to the parameters that define the centroid of a profile can be obtained from the formulae in Sect. A.1 via multiplication with a constant factor:

$$\begin{aligned}
\partial_\nu M &= \frac{-\beta\cdot I\cdot(\beta-1)}{\pi r_\mathrm{d}^2\cdot(1-e)}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-\beta-1}\frac{2r}{r_\mathrm{d}^2}\cdot\partial_\nu r \\
&= -\frac{2M\cdot\beta\cdot r}{r_\mathrm{d}^2}\left[1+\left(\frac{r}{r_\mathrm{d}}\right)^2\right]^{-1}\cdot\partial_\nu r\,, \qquad (35)
\end{aligned}$$

where again $\nu \in \{x^n, y^n, A, B, C, D\}$.